Exposing the Performance Hidden Costs: An Empirical Study of Application-Layer Overhead in High-Throughput gRPC Microservices

1st Irawan Widi Widayat
Department of Information and Intelligent Systems Engineering
Toyohashi University of Technology
Toyohashi, Japan
irawan@perf.cs.tut.ac.jp

2rd Irmawati
Department of Informatics and Computer Engineering
Politeknik Negeri Ujung Pandang
Makassar, Indonesia
irmawati@poliupg.ac.id

3nd Yukinori Sato
Department of Information and Intelligent Systems Engineering
Toyohashi University of Technology
Toyohashi, Japan
yukinori@cs.tut.ac.jp

4th Given Name Surname
dept. name of organization (of Aff.)
name of organization (of Aff.)
City, Country
email address or ORCID

***Abstract*—Current development of microservice architectures heavily relies on API gateways and high-performance RPC frameworks like gRPC, the implementation details of the inter-service communication layer are often abstracted away in system design. Our paper empirically investigates the performance degradation caused by application-layer communication bottlenecks, with particular focus on gRPC connection management. We present a mathematical queueing model and algorithmic formalization to quantify the overhead of the connection-per-request compared to a persistent multiplexed channel. Our evaluation results in Kubernetes environment tests show that fixing the single implementation flaw can significantly reduce median latency by 97%—from over 7,000 ms to 150 ms under peak load—moving the performance bottleneck from the infrastructure gateway back to the compute backend.**

***Index Terms*—microservice, grpc, cloud-native, kubernetes, high performance computing.**

I. Introduction

The shift toward cloud-native microservices has made gRPC (gRPC Remote Procedure Calls), built on HTTP/2, a widely adopted standard for high-performance communication between services. In many deployments, an API gateway is used to mediate traffic between external clients and internal microservices. Commonly, theoretical architectural models often assume that gateway-to-backend communication achieves near-optimal protocol efficiency, practical implementations often incur substantial and less visible latency overhead [1]. On the one hand, communication overhead can contribute a significant portion of the total request latency, usually overshadowing actual business logic execution. On the other hand, gRPC is designed to mitigate this via HTTP/2 multiplexing, improper lifecycle management of connections remains a common pitfall.

One important but often overlooked implementation factor is how TCP/HTTP/2 connection lifecycles are handled at the application level. Since HTTP/2 is designed to multiplex multiple requests over a single long-lived TCP connection, establishing a new connection for each incoming request—a practice carried over from traditional HTTP/1.1 REST-based systems—directly conflicts with the protocol's design.

Similar challenges have been documented in the context of database connection pooling. Studies in [2] show that the overhead of the TCP three-way handshake and TLS negotiation in short-lived connections can degrade throughput by orders of magnitude. In cloud-native environments, this issue is exacerbated by network virtualization layers. As noted by [3], the interaction between application-layer protocols and container networking (e.g., CNI plugins in Kubernetes) introduces non-deterministic tail latencies, making the "connection-per-request" anti-pattern particularly toxic to system stability.

Our work builds upon the theoretical foundations of queueing theory applied to cloud computing. Previous research [4] has modeled cloud service performance using $M/M/1$ or $M/M/c$ queues; however, these models often assume a constant service time. We argue that implementation-level flaws, such as gRPC connection churn, convert what should be a constant-time operation into a variable, high-latency event, causing premature system saturation as utilization approaches the limits defined by Erlang's formulas [5].

In this paper, we formalize the latency and throughput cost of gRPC "connection churn" within an API gateway. We introduce a queueing-theoretic model to address the nonlinear increase in latency observed under high load. We compare the

This work is based on results obtained from the project, "Research and Development Project of the Enhanced infrastructures for Post-5G Information and Communication Systems" (JPNP20017), commissioned by the New Energy and Industrial Technology Development Organization (NEDO).

model with empirical results from a Kubernetes deployment, finding that scalability depends not only on the high-level architecture but also on effective connection management, as a critical aspect of microservice technology.

II. Mathematical Modeling of gRPC Connection Management

In order to comprehend the communication bottleneck, it is essential to break down the entire lifecycle of a request as it progresses through the API gateway. This analysis will help us identify the key stages and factors contributing to the bottleneck in the communication process.

Let T_{total} be the total end-to-end latency experienced by the client. We define T_{total} as:

$$T_{\text{total}} = T_{\text{http}} + W_{\text{queue}} + S_{\text{gateway}} + T_{\text{network}} \tag{1}$$

Where:

T_{http} : ingress HTTP processing time.
W_{queue} : queueing delay at the gateway's worker processes.
S_{gateway}: service time required by the gateway to establish and execute the downstream gRPC call.
T_{network}: physical network propagation delay and backend computation time (baseline constant $c \approx 3$ ms).

The critical variable in our study is $S_{gateway}$. We model this under two distinct operational paradigms.

A. Paradigm 1: Connection-Per-Request (CPR)

In the CPR anti-pattern, the gateway initializes a new gRPC channel for every incoming request. The service time S_{cpr} is defined as:

$$S_{cpr} = t_{tcp} + t_{http2} + t_{rpc} + t_{teardown} \tag{2}$$

Where t_{tcp} is the TCP 3-way handshake, t_{http2} is the HTTP/2 SETTINGS frame negotiation, t_{rpc} is the actual payload serialization/transmission, and $t_{teardown}$ is the TCP teardown (FIN/ACK). Because t_{tcp} and t_{http2} require multiple network round-trip times (RTTs) and significant CPU context switching, $S_{cpr} \gg t_{rpc}$.

B. Paradigm 2: Persistent Channel Reuse (PCR)

In the optimized PCR pattern, the gateway initializes a single, thread-safe gRPC channel during application startup. All incoming requests are multiplexed over this channel. The setup costs are amortized over millions of requests, approaching zero per request. Thus, the service time S_{pcr} simplifies to:

$$S_{pcr} = t_{rpc} \tag{3}$$

C. Queueing Theory and Non-Linear Saturation

The catastrophic latency spikes observed in empirical testing (e.g., jumping to 7,000 ms) cannot be explained by S_{cpr} alone. We must apply an $M/M/c$ queueing model, where incoming requests arrive at rate λ and are serviced by c gateway worker threads at rate $\mu = 1/S_{gateway}$.

The utilization of the gateway system is $\rho = \frac{\lambda}{c \cdot \mu}$. According to Little's Law [6] and standard queueing theory, the average queueing delay W_{queue} grows asymptotically as utilization $\rho \to 1$:

$$W_{queue} \propto \frac{\rho}{1 - \rho} \tag{4}$$

Because S_{cpr} is significantly larger than S_{pcr}, the service rate μ_{cpr} is artificially low. Therefore, under the CPR paradigm, the system reaches critical utilization ($\rho \to 1$) at a much lower arrival rate λ (e.g., 400 RPS). When λ exceeds this threshold (e.g., 600 RPS), W_{queue} approaches infinity, resulting in the massive latency spikes and connection timeouts observed in our baseline data.

III. Algorithmic Formalization

The mathematical models illustrate a significant change in the algorithmic behavior of the API gateway. To clarify this shift, we outline two approaches below that highlight the specific computational processes involved.

Algorithm 1: Connection-Per-Request (The Bottleneck) This algorithm demonstrates the highly inefficient $O(N)$ connection overhead, where N is the number of incoming requests.

Algorithm 1 API Gateway Request Handler (CPR Paradigm)

Require: req_number: Integer
Ensure: is_prime_result: Boolean

1: **function** HandleRequest(req_number)
2: *// Network IO 1: Establish TCP & HTTP/2*
3: $channel \leftarrow$ grpc.insecure_channel(SERVER_ADDRESS)
4: *// Memory Allocation: Create Stub*
5: $stub \leftarrow$ PrimeCheckerStub($channel$)
6: $request\_payload \leftarrow$ IsPrimeRequest(number $=$ req_number)
7: *// Network IO 2: Execute RPC*
8: $response \leftarrow stub$.Check($request\_payload$)
9: *// Network IO 3: Teardown TCP*
10: $channel$.close()
11: **return** $response.is\_prime$
12: **end function**

Algorithm 2: Persistent Channel Reuse (The Optimization) This algorithm utilizes the Singleton pattern for the channel and stub, reducing the connection overhead to $O(1)$ globally.

By shifting the $O(N)$ network initialization steps (Lines 2-3 in Algorithm 1) to an $O(1)$ global initialization step (Lines 1-2 in Algorithm 2), the application effectively eliminates T_{tcp} and T_{http2} from the critical path of the request lifecycle, validating our mathematical model where $S_{pcr} < S_{cpr}$.

IV. Methodology

To validate the proposed mathematical model and pinpoint accurately where latency overhead originates, we conducted controlled experiments in a containerized distributed setting. Our evaluation follows a component isolation strategy: we first

Algorithm 2 API Gateway Request Handler (PCR Paradigm)

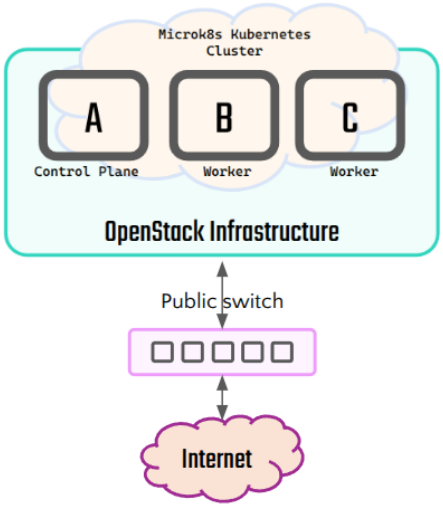
Require: req_number: Integer
Ensure: is_prime_result: Boolean

// Global Initialization (Executed Once at Startup)
$global\_channel \leftarrow$ grpc.insecure_channel(SERVER_ADDR
$global\_stub \leftarrow$ PrimeCheckerStub($global\_channel$)

function HandleRequest(req_number)
// Memory Allocation only
$request\_payload \leftarrow$ IsPrimeRequest(number $=$ req_number)

// Network IO 1: Execute multiplexed RPC instantly
$response \leftarrow global\_stub$.Check($request\_payload$)

return $response.is\_prime$
end function



- **Node:** 3 virtual machine nodes. 1 control plane node, 2 worker nodes.
- **Spec per node:** 2vCPU, 8GB-Memory, 30GB-Storage.
- **OS:** Ubuntu Linux 22.04 LTS Server.
- **Stack environment:** OpenStack.
- **K8s Cluster:** Microk8s.

Fig. 1. Kubernetes cluster environment.

measure end-to-end performance across the full system and then benchmark the backend service alone in order to distinguish computation time from communication-related delays.

A. Experimental Environment and Architecture

The experiment was conducted on a Kubernetes cluster composed of multiple worker nodes as shown in the Fig. 1. The system under test (SUT) consists of two main microservice components:

- **gRPC Backend Service (prime-server)**: A high-performance Python application responsible for CPU-intensive prime number verification. To eliminate backend capacity as a confounding variable, this component was over-provisioned with 10 identical replica pods, deployed behind a Kubernetes NodePort Service.
- **API Gateway (prime-frontend)**: A Python Flask application running within Gunicorn workers. This component acts as an HTTP/REST to gRPC proxy. It was strictly limited to 2 replica pods to intentionally force saturation and observe gateway-tier queuing dynamics under heavy load.

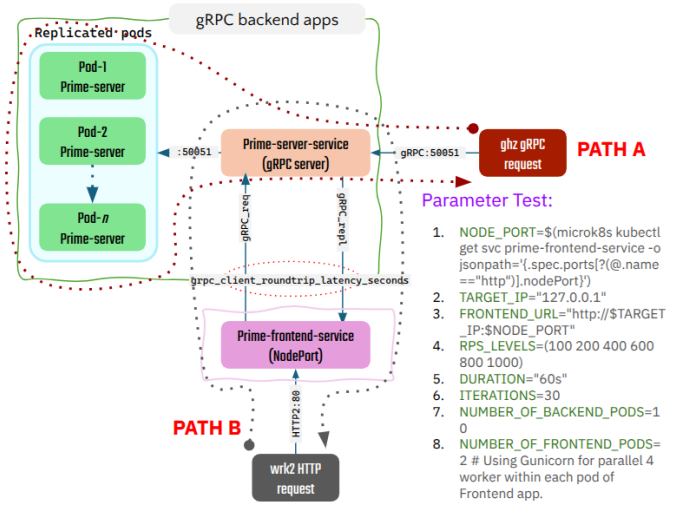


Fig. 2. Evaluation mechanism to accurately examine the frontend ($HTTP$) and backend ($gRPC$) latency utilizing different benchmark tools.

A strict Protocol Buffer ($prime.proto$) contract ensured type safety and rapid serialization/deserialization between the HTTP frontend and the gRPC backend.

B. Workload Generation and Component Isolation

To precisely calculate the application-layer gateway overhead, we utilized two specialized, open-source load-testing tools, as shown in the Fig. 2:

- **Isolated Backend Baseline** (ghz): To establish the absolute minimum latency of the business logic ($T_{network}$ in our mathematical model), we utilized ghz, a specialized gRPC benchmarking tool. Traffic was sent directly to the prime-server Service via NodePort, bypassing the frontend entirely.
- **End-to-End Measurement** ($wrk2$): To measure the total system latency experienced by a client (T_{total}), we utilized wrk2, an HTTP load-generation tool capable of maintaining a constant throughput rate. Traffic was directed to the prime-frontend pods. A dynamic Lua script was used to generate random payloads, preventing caching optimizations.

Both tools executed 30 iterations for each target throughput level (ranging from 100 to 1000 Requests Per Second, or RPS), sustaining the load for 60 seconds per iteration to allow queueing buffers to stabilize.

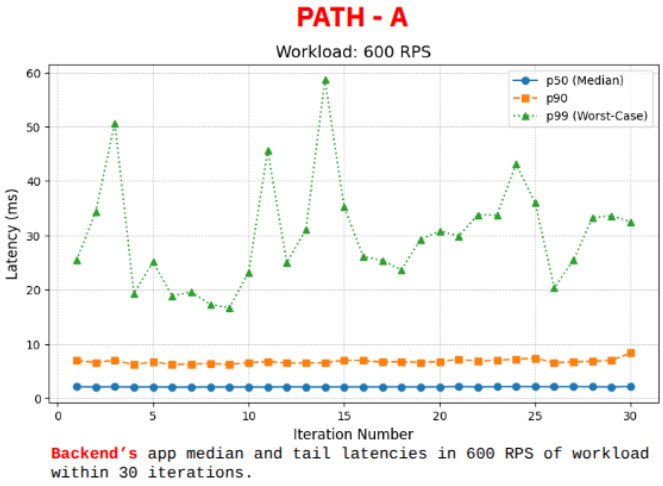


Fig. 3. gRPC backend latency evaluation using ghz benchmark tool.

V. Results and Discussion

The empirical results have successfully proven that the predicted Asymptotic latency degradation is caused by the $M/M/c$ queuing model in the Connection-Per-Request (CPR) paradigm and have clearly demonstrated the transformational impact of Persistent Channel Reuse (PCR).

A. Baseline: Backend gRPC Performance

The ghz test results established an exceptional and highly stable baseline. Across the entire load spectrum (100 to 1000 RPS), the 10 prime-server backend pods consistently delivered a median (p50) latency of $\sim$ 2.5 ms to 3.6 ms, with p90 latencies remaining firmly under 10 ms.

This flat latency curve definitively proved that the backend architecture was not the system bottleneck, as shown in the Fig. 3. By having 10 backend pods, we mathematically guaranteed that the backend possessed vastly more computational capacity than the frontend could ever saturate. The computational capacity of the prime number verification logic was highly resilient, isolating the source of any overarching performance degradation to the gateway tier.

B. Unoptimized System: The CPR Bottleneck

Testing the end-to-end system utilizing the CPR paradigm revealed a catastrophic bottleneck, as shown in the Fig. 4-A (upper side). At a moderate load of 400 RPS, the system maintained a median latency of $\sim$ 400 ms. However, as throughput increased to 600 RPS, the system hit the critical utilization threshold ($\rho \to 1$).

Median latency exploded to $> 7,100$ ms, and p99 latency spiked past $14,000$ ms. This exponential degradation perfectly aligns with the queueing theory model where service time (S_{cpr}) is artificially inflated by excessive TCP/HTTP2 handshake overheads for every transaction. The gateway workers were entirely occupied managing network state rather than processing business logic.

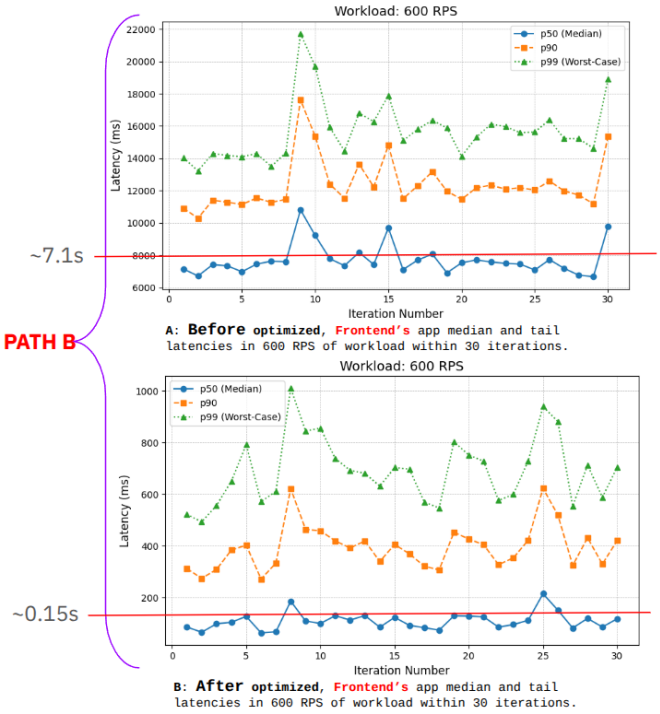


Fig. 4. End-to-end frontend latency evaluation using wrk2 as workload generator. (A) Connection-Per-Request paradigm - **Before** optimized, (B) Persistent Channel Reuse paradigm - **After** optimized

C. Optimized System: PCR Implementation

By deploying the algorithmic optimization detailed in Section III (establishing a persistent, multiplexed gRPC channel at application startup - Algorithm 2), the performance profile of the system was qualitatively transformed.

At 600 RPS, the optimized system recorded a median latency of $\sim$ 150 ms, as shown in the Fig. 4-B (lower side). This represents a $\sim$ 97.9% reduction in end-to-end latency compared to the CPR paradigm. Furthermore, the system successfully sustained loads up to 800 RPS before demonstrating signs of stress, effectively doubling the system's maximum throughput capacity without provisioning any additional infrastructure resources.

D. Correlating the "Red Dot Line" of Overhead

By subtracting the baseline backend latency ($T_{network}$) established via *ghz* from the full end-to-end system latency (T_{total}) measured via *wrk2*, we can isolate the inherent overhead of the API gateway—a metric we define as the "Red Dot Line" of application-layer tax.

As illustrated in Table I, the results reveal a compelling paradox in the optimized PCR paradigm. Although the persistent channel implementation achieved a 97% reduction in total latency compared to the CPR baseline, a compositional analysis shows that the gateway tier still accounts for **97.5% of the remaining end-to-end response time**.

This data suggests that the user experience is no longer bottlenecked by network state management (the connection

TABLE I
COMPARATIVE LATENCY OVERHEAD AT OPTIMAL SATURATION POINT (400 RPS)

Component Level	Median Latency (p50)	Percentage of Total Latency
Backend Only (*ghz*)	~ 2.8 ms	2.5%
Full System (CPR, Before)	~ 400 ms	100%
Full System (PCR, After)	~ 110 **ms**	**100%**
Gateway Overhead (PCR)	~ 107.2 **ms**	**97.5%**

churn), but rather by the **structural overhead of the proxying logic** itself. In our PCR model, the actual "work"—calculating prime numbers—takes only 2.8 ms (2.5% of total time), while the act of receiving, parsing, and forwarding the request through the Python/Flask/Gunicorn stack consumes 107.2 ms.

Critically, from the perspective of the end-user, the experience remains suboptimal. While we successfully averted a total system collapse, the user still perceives a "slow" interface where over 97% of their wait time is attributed to the internal machinery of the gateway rather than the service they requested. We argue that this represents a fundamental "Performance-Abstraction Paradox" in cloud-native design: as developers utilize high-level frameworks for rapid deployment, they inadvertently introduce a "tax" that can be orders of magnitude larger than the business logic itself. This finding shifts the research focus from protocol optimization to framework selection, suggesting that for high-throughput gRPC ecosystems, general-purpose WSGI frameworks may be architecturally unsuitable for providing a truly low-latency user experience.

VI. CONCLUSION AND FUTURE WORK

This study provides a rigorous, empirical demonstration of how subtle application-layer implementation details can catastrophically undermine scalable, cloud-native architectures. By formalizing the cost of gRPC connection churn, we quantified that a single architectural anti-pattern—creating new connections per request—was responsible for reducing system throughput by 50% and increasing latency by over 7,000 ms.

While optimizing to a persistent channel pattern (PCR) resolved the immediate queueing collapse, our component isolation methodology revealed that the application-layer proxy inherently adds ~ 100 ms of overhead. The primary conclusion drawn from this data is that implementation correctness is as vital as architectural scale, and that infrastructure concerns (such as API routing) should not be handled by general-purpose application frameworks.

Future Work: The logical progression of this research is twofold. First, to completely eliminate the identified "Red Dot Line" of overhead, future studies should evaluate migrating the API gateway functionality from a Python application to a highly optimized, compiled infrastructure component, such as an NGINX Ingress Controller or Envoy proxy. Second, future work will expand this steady-state benchmarking methodology into the domain of Chaos Engineering. By introducing controlled faults (e.g., node resource starvation, sudden pod termination) under a persistent load, researchers can quantify the specific latency cost of Kubernetes' self-healing mechanisms and define formal metrics for graceful degradation in distributed systems.

ACKNOWLEDGMENT

REFERENCES

[1] A. Abhishek, "Solving Espresso's scalability and performance challenges to support our member base," LinkedIn Engineering Blog, Sept, 7. 2023. [Online]. Available: https://www.linkedin.com/blog/engineering/infrastructure/solving-espresso-s-scalability-and-performance-challenges-to-sup

[2] I. Kasun, K. Danesh, gRPC: Up and Running: Building Cloud Native Applications with Go and Java for Docker and Kubernetes. O'Reilly Media, 2020.

[3] S. Shah, and M. Dubaria, "A Survey of Benchmarking Tools for Kubernetes Networking," 2019 IEEE 12th International Conference on Cloud Computing (CLOUD), pp. 500–505, 2019.

[4] P. Thiago, M. Marco, P. Paulo, L. Luan, S. Daliton, D. Jamilson, and M. Paulo, "Performance Modeling of Microservices with Circuit Breakers using Stochastic Petri Nets," 2024 IEEE International Systems Conference (SysCon), Montreal, QC, Canada, 2024, pp. 1-8, DOI: 10.1109/SysCon61195.2024.10553490.

[5] H. Robert, van H. André, K. Holger, L. Fei, L Lucy, Ellen, P. Claus, S. Stefan, and W. Johannes, "Performance engineering for microservices: Research challenges and directions," Proceedings of the 8th ACM/SPEC on International Conference on Performance Engineering Companion, pp. 223–226, 2017.

[6] Little, J.D., "Or forum—Little's law as viewed on its 50th anniversary. Operations research," 59(3), pp.536-549, 2011.